Replicating and Extending LLM-based Privacy Leakage Detection in Android App Logs

Cora Snyder, Peyton Boggs
cesnyder01@wm.edu, prboggs@wm.edu

May 7 2026

Abstract

Android application logs are an indispensable tool for developers, but they frequently capture sensitive user information that can be exploited by malicious actors. Chen et al. [7] recently demonstrated that large language models, specifically ChatGPT, can be applied to detect privacy leakage in such logs, with performance improving substantially when prompts are enriched with contextual knowledge (context-priming). However, their study was limited to a single LLM and has not yet been independently replicated. In this work we (i) replicate the two core research questions from Chen et al. using ChatGPT-4o mini and (ii) extend the study to Claude, assessing whether an alternative frontier LLM yields superior baseline or context-primed detection ability. Because neither author possesses API access, we conduct a manual evaluation across a curated sample of 100 representative Android log entries selected to cover a diverse range of true-positive, false-positive, and ambiguous cases. We present the complete prompt designs, annotated log corpus, and annotation protocol, and we discuss implications for privacy-aware log management and future automated pipelines.

1 Introduction

Android applications generate rich runtime logs that developers use for debugging, performance monitoring, and system telemetry [7]. These logs are captured by the Android system logging framework (logcat) and can be read by privileged applications or transmitted to remote analytics servers. While Google has restricted log access since Android 4.1, heterogeneous enforcement across the fragmented Android device ecosystem means that an attacker who gains physical or software access to a device can often retrieve application logs [7, **?**].

The privacy implications of this access are significant. Mobile application logs can contain personally identifiable information (PII) such as names, email addresses, telephone numbers, GPS coordinates, and device identifiers, as well as behaviorally sensitive data such as search queries, location histories, and usage patterns [**?**]. Once exfiltrated, these logs can facilitate targeted advertising, social engineering, identity theft, and physical surveillance.

Prior approaches to detecting privacy leakage in logs have relied on regular expressions and keyword lists [24]. These methods suffer from two key limitations: (i) they fail to generalize beyond predefined leakage patterns, and (ii) they cannot distinguish contextually sensitive from benign appearances of the same string. For example, the email address support@company.com appearing in a log line should not be treated the same as a user's personal email address synced from their contacts application.

The emergence of large language models (LLMs) such as ChatGPT and Claude opens new possibilities for context-aware privacy leakage detection. Chen et al. [7] published an empirical study demonstrating that ChatGPT can detect privacy leakage in Android app logs both without any additional guidance (baseline) and with significantly higher accuracy when provided with examples of verified privacy information types and values from the dataset (context-priming). Their RQ2 results are striking: context-priming increased ChatGPT's true positive count from 137 to 5,006 detections while simultaneously reducing false positives from 32,626 to 616, yielding a privacy-leak rate of 91.79%.

Despite these promising results, two questions remain open. First, the Chen et al. results have not been independently replicated: the same pipeline could yield different outcomes due to model version differences, prompt sensitivity, or annotation disagreements. Second, and perhaps more importantly, Chen et al. evaluated only ChatGPT. Whether Claude, a frontier LLM with different training objectives and safety properties, exhibits comparable or superior detection ability is unknown.

This paper addresses both gaps. Our contributions are:

1. Replication: We repeat the Chen et al. pipeline on ChatGPT-4o mini using a manually curated sample

of ten Android log lines, replicating their baseline (RQ1) and context-primed (RQ2) conditions.

2. Extension: We apply the identical evaluation to Claude (claude-sonnet-4-6), providing the first direct comparison of the two leading frontier LLMs on this task.

3. Detailed design disclosure: We publish the exact prompts, annotated sample log corpus, and annotation decision criteria used in our experiments, enabling future replication and extension.

4. Methodology adaptation: Operating without API keys, we conduct manual web-interface evaluations, and we discuss the implications and limitations of this constraint for reproducibility.

The remainder of this paper proceeds as follows. Section 2 overviews our sample paper. Section 3 describes the design of our sample paper. Section 4 evaluates our solution. Section 5 discusses additional topics. Section 6 describes related work. Section 7 concludes.

2 Overview

This paper presents a replication and extension of Chen et al.'s 2026 empirical study on using large language models to detect privacy leakage in Android application logs. The original study demonstrated that ChatGPT could identify sensitive user information embedded in app logs — such as email addresses, GPS coordinates, device identifiers, and IP addresses — and that its accuracy improved dramatically when prompts were enriched with verified examples of privacy information drawn from the dataset, a technique the authors call context-priming. Despite these promising results, the study evaluated only one LLM and its findings have not been independently confirmed.

We address two gaps. First, we replicate the original experiment's two core research questions using ChatGPT-4o mini to determine whether the same qualitative and quantitative patterns hold under independent evaluation. Second, we extend the study to Claude (claude-sonnet-4-6), providing the first direct comparison of two leading frontier LLMs on this task. Because neither author has API access, we adapt the methodology to a manual evaluation framework: rather than submitting large batches of log files programmatically, we curate a 100-entry sample corpus drawn from the original 300-app dataset, selecting entries that collectively cover every privacy type in the original taxonomy alongside canonical false-positive and ambiguous cases.

The study is organized around five research questions. RQ1 and RQ2 ask whether ChatGPT's baseline behavior, including its tendency toward format-driven false positives, and its improvement under context-priming can be replicated. RQ3 through RQ5 ask whether Claude exhibits superior baseline detection ability, whether it also benefits from context-priming, and whether context-primed Claude outperforms context-primed ChatGPT.

Beyond replicating experimental outcomes, this paper's primary contribution is methodological transparency. We publish the verbatim prompts used in both the baseline and context-primed conditions, the full annotated log corpus with ground-truth labels, and the explicit annotation decision criteria used to resolve ambiguous cases — materials that the original study did not make available.

2.1 Threat Model

We adopt the threat model established by Chen et al. [7], which we summarize and extend here for completeness."),

Attacker capabilities: The attacker has obtained Android application log files through any available vector, e.g., a malicious application running on the device, a compromised third-party analytics library, physical access to an unlocked device, or exfiltration during transmission to a remote server. The attacker also has access to a frontier LLM (ChatGPT or Claude) via a standard chat interface. No API key is assumed: all interactions occur through the publicly available free or subscription web interfaces.

Attacker goals: The attacker seeks to extract as much personally identifiable or behaviorally sensitive information as possible from the log files. Specific targets include: PII (names, email addresses, phone numbers), location data (GPS coordinates, city-level location), device identifiers (manufacturer, model, hardware serial), IP addresses, and usernames.

Attacker limitations: The attacker does not manually inspect the log files; they rely entirely on LLM-generated summaries and detections. They have no access to the Android device, its network traffic, or any external database. The attacker operates in a closed environment, having already downloaded the log files and gained access to the LLM.

Scope: The mechanism of log acquisition is out of scope. We focus exclusively on the post-acquisition analysis phase: given a set of log files, can an LLM identify privacy-leaking entries? This scoping decision is identical to that of Chen et al. [7].

This threat model reflects realistic attacker scenarios. Embedded SDKs such as advertising and analytics libraries routinely transmit log data to third-party servers with limited developer oversight. If such a third-party actor were adversarial, they could apply exactly this pipeline to the logs they receive.

2.2 Research Questions

We organize our study around five research questions. RQ1 and RQ2 replicate the core questions from Chen et al. [1]. RQ3 through RQ5 extend the study to Claude.

1. RQ1: Will ChatGPT demonstrate a measurable baseline ability to identify privacy leakage in Android app logs, in particular, exhibiting the high false-positive pattern attributed to format-matching — consistent with the findings of Chen et al. [7]?

2. RQ2: Does context-priming ChatGPT substantially increase its true-positive detection rate and reduce false positives, replicating the improvement observed by Chen et al. [7]?

3. RQ3: Does Claude exhibit a higher baseline accuracy (higher TP rate, lower FP rate) in detecting privacy leakage compared to ChatGPT's baseline under equivalent conditions?

4. RQ4: Does Claude's detection accuracy improve when context-primed, analogously to the improvement observed for ChatGPT?

5. RQ5: With context-priming, is Claude superior to ChatGPT in both quantity of true privacy leakage detected and precision (lower false-positive rate)?

3 Design

This section is the primary contribution beyond Chen et al. [7]. We describe in full detail the dataset, the log sample, the exact prompts, the annotation criteria, and the adaptations necessitated by operating without API access.

3.1 Dataset

We adopt the same dataset as Chen et al. [7]: 300 Android application log files captured from popular applications drawn from the AndroZoo repository, collected on a Google Pixel 5a running Android via the Android Debug Bridge (ADB). Each application was executed for approximately 30 minutes under simulated user interaction, yielding a total of 8,032,477 log records across the 300 files. This dataset is publicly available via the project GitHub repository cited in the original paper.

Because we cannot batch the full 300-file corpus through an LLM without API access, we curated a representative sample of 100 log entries, reflective of the Chen et al. dataset (Section 5.3). These entries were chosen to ensure coverage of each privacy information type identified in the original study, including true-positive, false-positive, and ambiguous other cases.

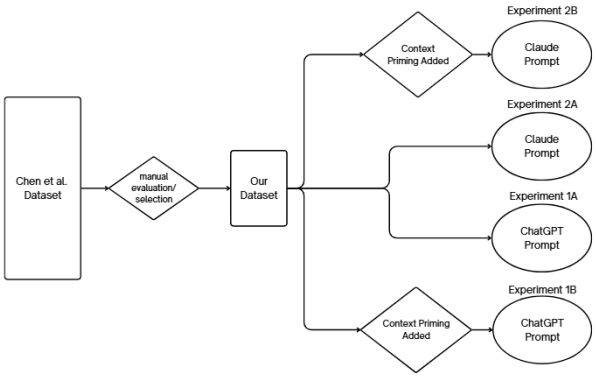


Figure 1: Prompt Creation Process

3.2 Experimental Prompts

Chen et al. [7] published the verbatim prompts on GitHub, and so we precisely replicate them here.

3.2.1 Baseline Prompt

The following prompt was used for the baseline (no context-priming) condition, submitted to both ChatGPT and Claude. The placeholder [LOG FILE CONTENT] was replaced with the literal log entries, one per line.

> " The file is an Android log file and I need you to analyze each log line in the file to find out if it contains private information.
>
> The output file has three requirements:
>
> 1. The output file should be in tsv format.
>
> 2. The output file has 5 columns, the first row should be ID, Sentence, Value, Description, Type.
>
> 3. The following are the requirements for each column. The ID column is the original order of each log line. The Sentence column is the original log line. The Value column has two different values, 1 and 0. 1 means that this log line contains private information and 0 means that this log line does not contain private information. The Description column is if this log line contains private information and the corresponding value column is 1, list what you think is private information in this cell, just copy the private information and do not modify the original information. The Type column is the type of privacy information.
>
> [LOG FILE CONTENT] "

3.2.2 Context-Primed Prompt

The context-primed prompt incorporates verified examples of privacy information types and values from the dataset. It includes the same thesis statement and first three requirements, and gives context in the fourth requirement. As with the baseline prompt, [LOG FILE CONTENT] was replaced with the actual log entries.

> "[ORIGINAL PROMPT]
>
> The following is any potentially private information we have detected that you think you can list in addition to this information.
>
> And maybe some privacy is incomplete in the log line. For example, Google Pixel 5a may show in the log line as Pixel 5a or Pixel, we still think it is a privacy. GPS is another example, GPS's beginning with 43 and -77 should also be considered private.

Model	Pixel 5a
Info type	Info
Android ID	24214e943...
Email	rit...23@gmail.com
Email	rit...135@gmail.com
ID (SHA-1)	b483ee136544...
Serial	17061JECB05867
ID (MD5)	997ec837e4a9...
Serial (MD5)	92b7fc65e0...
Name	rit research
IMEI	357641622194041
GPS	43.0838,-77.6799
GPS	43.xxxx,-77.xxxx
Fine location	Rochester
Course location	New York
IP	68.180.86.24

3.3 Annotation

Our annotation protocol follows the two-phase approach of Chen et al. [7]. In the first phase, annotators verify whether the element flagged by the LLM genuinely matches the claimed privacy type (e.g., does the flagged string actually constitute an IP address, and is it a real, routable one?). In the second phase, annotators verify whether that element constitutes actual privacy leakage (e.g., is the IP address attributable to the user's device rather than a third-party server?)

3.4 Methodology

We conduct four experiments organized in a 2×2 design crossing LLMs (ChatGPT, Claude) with context-priming condition (baseline, primed). This structure directly addresses all five research questions.

3.4.1 Experiment 1a (RQ1)

Procedure. Each log will be submitted to ChatGPT with the original prompt instructing it to identify privacy leakage line by line. No additional context or domain knowledge will be provided, so as to measure the model's baseline detection ability. The results will then be annotated.

Hypothesis. ChatGPT will demonstrate a measurable baseline ability to identify privacy leakage in Android app logs without context priming, consistent with the findings of Chen et al. Specifically, we expect ChatGPT to exhibit a high false positive rate due to its reliance on format-matching rather than contextual understanding of privacy information.

3.4.2 Experiment 1b (RQ2)

Procedure. The same prompting procedure from Experiment 1a will be used, with the addition of the context priming material.

Hypothesis. Providing ChatGPT with contextual knowledge about privacy leakage types and representative examples (context priming) will substantially increase its true positive detection rate and reduce false positives, consistent with the RQ2 findings of Chen et al.

3.4.3 Experiment 2a (RQ3)

Procedure. The log files will be submitted to Claude. Claude will be prompted with an equivalent concise instruction to identify privacy leakage line by line, with no additional context provided. The same annotation process from Experiments 1a/1b will be applied.

Hypothesis. Claude will demonstrate a higher baseline accuracy in detecting privacy leakage in Android app logs compared to ChatGPT's baseline.

3.4.4 Experiment 2b (RQ4 and RQ5)

Procedure. Claude will be submitted the same log files with the context priming material appended to the detection prompt, mirroring the Experiment 1b procedure. The same annotation process will be applied.

Hypothesis (RQ4). Applying context priming to Claude will increase its privacy leakage detection efficacy, analogous to the improvement observed for ChatGPT in Experiment 1b.

Hypothesis (RQ5). With context priming, Claude will outperform context-primed ChatGPT (Experiment 1b) in both the quantity of true privacy leakage detected and in precision (lower FP rate).

4 Evaluation

We evaluated the performance of the LLMs by calculating the total True Positives, True Negatives, False Positives, and False Negatives generated by each. From these values we calculated the accuracy of each.

4.1 ChatGPT Baseline Results

	Positive	Negative
True	30	36
False	15	19

Accuracy: 66% **Precision:** 66.7%
Recall: 61.22% **F1 Score:** 63.83%

4.2 Claude Baseline Results

	Positive	Negative
True	49	24
False	27	0

Accuracy: 73% **Precision:** 64.47%
Recall: 100% **F1 Score:** 78.4%

4.3 ChatGPT Primed Results

	Positive	Negative
True	49	9
False	42	0

Accuracy: 58% **Precision:** 53.85%
Recall: 100% **F1 Score:** 70%

4.4 Claude Primed Results

	Positive	Negative
True	48	43
False	8	1

Accuracy: 91% **Precision:** 85.71%
Recall: 97.96% **F1 Score:** 91.43%

4.5 Interpretation

ChatGPT's accuracy and precision scores decreased after context priming, due to a significant increase in false positives. This suggests that over-priming may be a concern. However, its recall and F1 scores increased, due to a significant reduction in false negatives. [Taken together, these results constitute a partial replication of Chen et al.: the qualitative recall improvement is consistent with the original findings (RQ2), but the precision collapse represents a meaningful divergence, possibly attributable to model version differences or the smaller sample size used here.

Claude's results followed the pattern observed in Chen et al.'s original experiment: the LLM scored fairly well on the baseline evaluation, then improved after context priming. Interestingly, there was a slight decrease in the recall score. It appears that this reduction was the tradeoff for the increase in accuracy, precision, and F1 scores. Context-primed Claude also substantially outperformed context-primed ChatGPT on accuracy (91% vs. 58%), precision (85.71% vs. 53.85%), and F1 (91.43% vs. 70%), supporting RQ5.

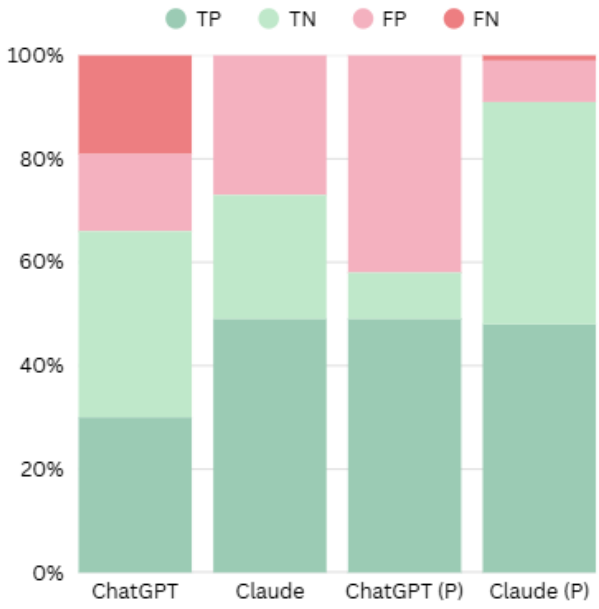


Figure 2: Overall Results

5 Discussion

Several simplifying assumptions constrain the interpretation of our results. Our evaluation relies on a manually curated sample of 100 log entries rather than the full 300-application corpus, limiting statistical reliability. Additionally, all LLM interactions were conducted manually through web interfaces rather than via API, introducing potential variability between sessions and reducing reproducibility.

Our results also raise questions about the replicability of context-priming findings across model versions and sample compositions. Where Chen et al. observed dramatic improvements in both true positive rate and precision under context-priming, our replication found that priming improved ChatGPT's recall at the cost of a significant increase in false positives, ultimately reducing overall accuracy. This may reflect differences in model version — we used GPT-4o mini rather than the version evaluated by Chen et al. — or the effect of sample size, where a smaller corpus amplifies the impact of each additional false positive on calculated metrics. Notably, Claude did

not exhibit this same vulnerability to over-priming, suggesting that the two models respond differently to the same priming strategy. This asymmetry shows that practitioners cannot assume that a validated priming approach on one LLM will transfer cleanly to another, and replication across model versions should be considered a standard practice.

Future studies should consider evaluating LLMs in detecting specific types of sensitive data, or in specific circumstances or use cases. The LLMs, while effective, produced very different results, suggesting that they may have different strengths which could be capitalized on once fully explored. In particular, Claude's avoidance of false negatives — at the cost of more false positives — makes it preferable in high-stakes settings such as medical or legal applications, where a missed privacy leak carries serious consequences. ChatGPT's more conservative flagging behavior, with its lower false-positive rate at baseline, may be better suited to high-volume contexts where analyst bandwidth is limited and precision matters more than exhaustive recall. Future work should evaluate both models on domain-specific use cases where the relative cost of each error type is well-defined. More broadly, the fact that general-purpose LLMs with no task-specific training can perform with a baseline accuracy of above 60% on this problem suggests that privacy-aware log management does not require expensive specialized tooling — a finding with practical implications for smaller development teams who lack the resources to deploy dedicated static analysis pipelines.

6 Related Work

This work sits at the intersection of two particularly active research areas: privacy leakage in Android application logs and the usage of LLMs in cybersecurity. Because of this, there are plenty of previous papers that have influenced this work.

Nam et al. [22] demonstrated that LLMs can explain and reason about code with accuracy comparable to senior developers, establishing their viability for the hybrid natural-language-and-code domain that Android logs occupy. Niu et al. [23] applied this to vulnerability detection specifically, generating an empirical study mirroring the results of Chen's. Liu et al. [19] provide broader theoretical grounding for this pattern in their survey of prompting methods, and finally, Hou et al. [14] offer a comprehensive review of LLMs across software engineering tasks, contextualizing privacy leakage detection as one of many domains where LLMs show measurable but improvable utility.

In recent years, there has been increased interest in LLMs for analysis and detection. Research in this arena ranges from the application of LLMs for anomaly detection in data [17] to LLMs' ability to distinguish between genuine privacy-sensitive log content from benign records [27]. Before the boom of LLMs using transformers, Harkous et al. [11] developed a neural framework called Polisis in 2017 that demonstrated how models can reliably extract data practice information from privacy policies. Two years later, now equipped with GPT-class models, Ravichander et al. confirmed and extended these results [26]. The in-context learning capabilities that made such improvements possible were formally characterized by Brown et al. [3] with GPT-3, establishing the few-shot prompting paradigm that underlies the contextual priming approach evaluated in this work. Additionally, LLMs have been found particularly effective at log analysis, which was examined in depth by Akhtar et al. [1].

As LLMs gained popular traction, their use for privacy and security detection was landscaped by Yao et al. [29] in 2023. Their research centered on ChatGPT and Bard, and highlighted the benefits, drawbacks, and vulnerabilities of using such models for cybersecurity. Specifically, research has recognized the potential of using ability of LLMs to analyze unstructured data in large quantities for cybersecurity. Palma et al. [25] found that LLMs outperformed traditional machine learning models in the analysis of complex cybersecurity logs.

System logs appear in a variety of formats and provide component-specific system-state information, which has a variety of purposes [24]. Unfortunately, the possibility of sensitive data being recorded in logs poses a major cybersecurity risk to users [21]. Although focused on web query logs rather than system logs, Jones et al. [15] demonstrated the privacy risks inherent in log data; using information extracted from web query logs, they showed that a user's identity could be narrowed down to a set 300–600 times smaller than random chance. Research has attempted to identify and correct privacy leakage in logs [8, 16]. Mobile application logs in particular have been found to pose a cybersecurity risk due to the volume and type of data involved [31]. Analysis by Harty et al. [12] determined that compared to non-mobile application logging practices, mobile app logs center more on recording user data, rather than recording information for debugging purposes.

The field of software log analysis more broadly informs the detection pipeline used here. Oliner et al. [24] provided an early 2012 survey of the challenges in log analysis, establishing log parsing and anomaly detection as core tasks. He et al. [13] addressed the parsing problem with Drain, an efficient online algorithm that extracts log templates from data. Chen and Jiang [4] later identified developers frequently and inadvertently logging sensitive variables, framing privacy leakage as a software engineer-

ing problem rather than purely a security one. In a survey of developers' perspectives of the benefits and costs of logging practices, only 2 out of 66 survey respondents listed exposing sensitive user information as a cost [18].

The role of software engineering in log privacy leakage has been confirmed in studies of privacy leakage in Android applications at the system level, an area which has been researched extensively. Examining 67,702 logs from 83 Android apps, Chen et al. [6] found 610 instances of privacy leakage. Enck et al. [9] introduced TaintDroid, a dynamic taint-tracking framework that revealed how sensitive data flows without user awareness. Arzt et al. [2] followed with FlowDroid, a static analysis tool applying taint analysis to Android bytecode, though it remained constrained by predefined source-sink specifications. Felt et al. [10] conducted an empirical analysis of Android permission systems, revealing widespread over-privileging and identifying a gap between permissions declared by applications and those actually required for their functionality. Lyons et al. [20] measured the logging of sensitive information across 230 Android applications and identified over 1,000 instances of personal data written to system logs. In a survey of sensitive data transmissions in Android apps, Yang et al. [28] found that sensitive data logged included device IDs, phone numbers, locations, and user contacts. In their evaluation of 693 Android banking apps around the world, Chen et al. [5] identified logging as a key security weakness in several. Zhao et al. [30] specifically examined the role of third party logging libraries in instances of android log privacy leakage.

7 Conclusion

This study set out to ask one question — can LLMs detect privacy leakage in Android logs — and arrived at another: not whether LLMs can do this, but which failure mode is more acceptable given the context. Both LLMs showed an ability to detect data leakage in Android app logs, with and without priming. This success suggests that the barrier to privacy auditing is lower than prior regex- and keyword-based approaches implied. However, our results also indicate that the LLMs may be better suited for different use cases. Claude tended away from false negatives; this may be better for use cases in which false negatives are more harmful than false positives. Conversely, ChatGPT's lower false-positive rate at baseline may be preferable in high-volume settings where precision and analyst efficiency are the priority. In terms of log analysis, this would suggest that Claude would be best utilized in cases of highly confidential data use, where the cost of missing a data leak is greater than the cost of investigating a false positive. What remains unresolved is whether either model's behavior is stable across log formats, application domains, and prompt variations — or whether the results here are artifacts of this particular sample. The deeper implication is that deploying LLMs for privacy auditing may require not just choosing a model, but choosing a failure mode, and that choice should be intentional.

References

[1] S. Akhtar, S. Khan, and S. Parkinson. Llm-based event log analysis techniques: A survey, 2025.

[2] S. Arzt, S. Rasthofer, C. Fritz, E. Bodden, A. Bartel, J. Klein, Y. Le Traon, D. Octeau, and P. McDaniel. FlowDroid: Precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for Android apps. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 259–269, Edinburgh, UK, 2014.

[3] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. Language models are few-shot learners. In *Proceedings of the 34th Conference on Neural Information Processing Systems (NeurIPS)*, pages 1877–1901, Online, 2020.

[4] B. Chen and Z. M. Jiang. Characterizing and detecting anti-patterns in the logging code. In *Proceedings of the 39th International Conference on Software Engineering (ICSE)*, pages 71–81, Buenos Aires, Argentina, 2017.

[5] S. Chen, L. Fan, G. Meng, T. Su, M. Xue, Y. Xue, Y. Liu, and L. Xu. An empirical assessment of security risks of global android banking apps. In *2020 IEEE/ACM 42nd International Conference on Software Engineering (ICSE)*, pages 1310–1322, 2020.

[6] Z. Chen, S. S. Deo, P. C. R. Puttaparthi, Y. Tang, X. Zhang, and W. Shang. From logging to leakage: A study of privacy leakage in android app logs. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, ASE '24, page 2484–2485, New York, NY, USA, 2024. Association for Computing Machinery.

[7] Z. Chen, V. Nava-Camal, T. Roy, Z. Li, Y. Tang, X. Zhang, and H. Yang. Exploring large language models' potential for privacy leakage detection in android app logs: An empirical study. *IEEE software*, 43(1):57–63, 2026.

[8] A. Cooper. A survey of query log privacy-enhancing techniques from a policy perspective. *ACM Trans. Web*, 2(4), Oct. 2008.

[9] W. Enck, P. Gilbert, S. Han, V. Tendulkar, B.-G. Chun, L. P. Cox, J. Jung, P. McDaniel, and A. N. Sheth. TaintDroid: An information-flow tracking system for realtime privacy monitoring on smartphones. *ACM Transactions on Computer Systems*, 32(2):1–29, June 2014.

[10] A. P. Felt, E. Ha, S. Egelman, A. Haney, E. Chin, and D. Wagner. Android permissions: User attention, comprehension, and behavior. In *Proceedings of the 8th Symposium on Usable Privacy and Security (SOUPS)*, pages 1–14, Washington, DC, USA, 2012.

[11] H. Harkous, K. Fawaz, R. Lebret, F. Schaub, K. G. Shin, and K. Aberer. Polisis: Automated analysis and presentation of privacy policies using deep learning. In *Proceedings of the 27th USENIX Security Symposium (USENIX Sec.)*, pages 531–548, Baltimore, MD, USA, 2018.

[12] J. Harty, H. Zhang, L. Wei, L. Pascarella, M. Aniche, and W. Shang. Logging practices with mobile analytics: An empirical study on firebase. In *2021 IEEE/ACM 8th International Conference on Mobile Software Engineering and Systems (MobileSoft)*, pages 56–60, 2021.

[13] P. He, J. Zhu, Z. Zheng, and M. R. Lyu. Drain: An online log parsing approach with fixed depth tree. In *Proceedings of the IEEE International Conference on Web Services (ICWS)*, pages 33–40, Honolulu, HI, USA, 2017.

[14] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology*, 33(8):1–79, Nov. 2024.

[15] R. Jones, R. Kumar, B. Pang, and A. Tomkins. "i know what you did last summer": query logs and user privacy. In *Proceedings of the Sixteenth ACM Conference on Conference on Information and Knowledge Management*, CIKM '07, page 909–914, New York, NY, USA, 2007. Association for Computing Machinery.

[16] J. Lee, Y. Lee, M. Jin, J. Kim, and J. Hong. Analysis of application installation logs on android systems. In *Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing*, SAC '19, page 2140–2145, New York, NY, USA, 2019. Association for Computing Machinery.

[17] A. Li, Y. Zhao, C. Qiu, M. Kloft, P. Smyth, M. Rudolph, and S. Mandt. Anomaly detection of tabular data using llms, 2024.

[18] H. Li, W. Shang, B. Adams, M. Sayagh, and A. E. Hassan. A qualitative study of the benefits and costs of logging from developers' perspectives. *IEEE Transactions on Software Engineering*, 47(12):2858–2873, 2021.

[19] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys*, 55(9):1–35, Jan. 2023.

[20] A. Lyons, J. Gamba, A. Shawaga, J. Reardon, J. Tapiador, S. Egelman, and N. Vallina-Rodriguez. Log: It's big, It's heavy, It's filled with personal data! measuring the logging of sensitive information in the android ecosystem. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 2115–2132, Anaheim, CA, Aug. 2023. USENIX Association.

[21] MITRE Corporation. CWE-532: Information Exposure Through Log Files. `https://cwe.mitre.org/data/definitions/532.html`, 2023. Accessed: 2026.

[22] D. Nam, A. Macvean, V. Hellendoorn, B. Vasilescu, and B. Myers. Using an LLM to help with code understanding. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*, pages 1–13, Lisbon, Portugal, 2024.

[23] Y. Niu, K. Li, J. Shi, X. Chen, and Y. Liu. Empirical study of LLM-based vulnerability detection with code context. In *Proceedings of the IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 1–12, Flagstaff, AZ, USA, 2024.

[24] A. Oliner, A. Ganapathi, and W. Xu. Advances and challenges in log analysis. *Communications of the ACM*, 55(2):55–61, Feb. 2012.

[25] G. Palma, G. Cecchi, M. Caronna, and A. Rizzo. Leveraging large language models for scalable and explainable cybersecurity log analysis. *Journal of Cybersecurity and Privacy*, 5(3), 2025.

[26] A. Ravichander, A. W. Black, T. Norton, S. Wilson, and N. Sadeh. Question answering for privacy policies: Combining computational and legal perspectives. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*

(EMNLP), pages 4947–4958, Hong Kong, China, 2019.

[27] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Proceedings of the 36th Conference on Neural Information Processing Systems (NeurIPS)*, pages 24824–24837, New Orleans, LA, USA, 2022.

[28] Z. Yang, M. Yang, Y. Zhang, G. Gu, P. Ning, and X. S. Wang. Appintent: Analyzing sensitive data transmission in android for privacy leakage detection. In *Proceedings of the 20th ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 1043–1054. ACM, 2013.

[29] Y. Yao, J. Duan, K. Xu, Y. Cai, Z. Sun, and Y. Zhang. A survey on large language model (llm) security and privacy: The good, the bad, and the ugly. *High-Confidence Computing*, 4(2):100211, 2024.

[30] Y. Zhao. An empirical study of privacy leakage vulnerability in third-party android logs libraries. Master of applied science thesis, University of Waterloo, Waterloo, Ontario, Canada, Oct. 2025.

[31] R. Zhou, M. Hamdaqa, H. Cai, and A. Hamou-Lhadj. Mobilogleak: A preliminary study on data leakage caused by poor logging practices. In *2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 577–581, 2020.